

Advanced R for Econometricians (Summer 2026)

Shiny — Notes and Examples

Martin Arnold, Martin Schmelzer

What is Shiny?

- Shiny apps run inside an R session on a server (or locally). The browser is just a thin client: it only renders HTML and fires events back to R.
- This is fundamentally different from a static R script. Execution is *event-driven*: code only runs in response to user actions or reactive invalidations.
- The three main use cases in practice:
 1. **GUI for R** — expose a model or analysis to a non-R audience.
 2. **Reporting** — interactive dashboards instead of static PDFs.
 3. **Interactive documents** — embed live widgets inside R Markdown or Quarto.
- Real-world deployment: Shiny apps are deployed via shinyapps.io or a self-hosted Posit Connect server.

Structure

- The split into `ui` and `server` is not cosmetic: it mirrors the browser–server boundary in web development.
- `ui` produces HTML that is sent to the browser. No R code runs inside `ui` at render time (except during initial startup).
- `server` is an ordinary R function. Shiny calls it once per user session and keeps it alive, allowing state to persist between interactions.
- The `session` argument gives access to session-level metadata and lets you programmatically update inputs from the server (`updateSliderInput()`, etc.).

A Minimal Example

```
library(shiny)

ui    <- fluidPage()           # the user interface
server <- function(input, output, session) {} # the R backend

shinyApp(ui = ui, server = server) # run the app
```

- `fluidPage()` returns a string of HTML. Type `fluidPage()` in the console to see it.
- `shinyApp()` bundles `ui` and `server` and returns an object of class "shiny.appobj". When printed interactively, it launches the app.
- **Two-file alternative:** some teams prefer the `ui.R` / `server.R` convention for larger apps because it separates concerns in the file system:

```
App-1/  
  ui.R
```

server.R

Launch with `shiny::runApp("App-1")`.

A third file `global.R` (if present) is executed once at startup and is visible to both `ui.R` and `server.R`.

User Interface

```
ui <- fluidPage(
  titlePanel("Title Panel"),
  sidebarLayout(
    sidebarPanel("Sidebar Panel"),
    mainPanel("Main Panel")
  )
)
server <- function(input, output, session) {}
shinyApp(ui = ui, server = server)
```

- `sidebarLayout()` is a convenience wrapper. Under the hood it uses Bootstrap's grid system (12 columns). `sidebarPanel` occupies 4 columns, `mainPanel` occupies 8.
- **See the generated HTML:** type any layout function in the console.

```
sidebarLayout(
  sidebarPanel("left"),
  mainPanel("right")
)
```

- Other layout options: `navbarPage()` (tabbed navigation), `fillPage()` (full-browser fill), `bootstrapPage()` (bare Bootstrap). For complex dashboards, the `bslib` package offers a modern theming system.

Adding Elements for User Input

```
ui <- fluidPage(
  titlePanel("Title Panel"),
  sidebarLayout(
    sidebarPanel("Sidebar Panel",
      br(),
      br(),
      sliderInput(
        inputId = "num",
        label = "Choose a number",
        value = 25, min = 1, max = 100)
    ),
    mainPanel("Main Panel")
  )
)
```

- Every input widget follows the same convention: the first argument is always `inputId`, the second is `label`. The rest are widget-specific.
- `inputId` must be unique within an app. It is the key used to access the value from the server via `input$inputId`.
- The value in `input$num` changes type depending on the widget: `sliderInput` returns a numeric, `textInput` returns a character, `checkboxInput` returns a logical.

- Overview of common input widgets:

Function	Returns	Typical use
<code>sliderInput()</code>	numeric / date	continuous ranges
<code>selectInput()</code>	character	choose from list
<code>checkboxInput()</code>	logical	on/off flag
<code>checkboxGroupInput()</code>	character vector	multiple choices
<code>radioButtons()</code>	character	mutually exclusive
<code>textInput()</code>	character	free text
<code>numericInput()</code>	numeric	exact number entry
<code>dateInput()</code>	Date	calendar picker
<code>fileInput()</code>	list	file upload
<code>actionButton()</code>	integer (click count)	trigger events

- A particularly useful widget for data exploration: `varSelectInput()` — it inspects a data frame and populates the choices automatically.

```
varSelectInput("xvar", "X variable", data = gapminder)
```

Create Output in the Server Function

```
server <- function(input, output, session) {
  output$hist <- renderPlot({
    hist(rnorm(input$num))
  })
}
```

- `input` and `output` behave like named lists but are reactive environments managed by Shiny.
- `renderPlot({...})` creates a *reactive expression* that re-executes whenever any `input$*` value it reads changes. The plot is re-rendered automatically.
- The `{}` inside render functions is an ordinary R code block. You can write multi-line expressions with intermediate variables.

```
output$hist <- renderPlot({
  n    <- input$num           # read reactive value
  data <- rnorm(n)            # compute
  hist(data, main = paste("n =", n))
})
```

- **Common mistake:** trying to read `output$hist` from the server. Output objects are write-only.

Display Output in the User Interface

```
ui <- fluidPage(
  titlePanel("Title Panel"),
  sidebarLayout(
    sidebarPanel(
      br(),
      sliderInput(inputId = "num",
                  label   = "Choose a number",
                  value   = 25, min = 1, max = 100)
    ),
  ),
```

```

mainPanel(
  br(),
  plotOutput("hist")
)
)
)

```

- `plotOutput("hist")` places an `<img>` placeholder in the HTML. The "hist" string must exactly match the `output$hist` assignment in the server.
- The string matching between `renderPlot` → `output$hist` → `plotOutput("hist")` is Shiny's wiring convention. A mismatch produces no output and no error — easy to miss.

Preventing Errors with `req()`

```

server <- function(input, output, session) {
  output$hist <- renderPlot({
    req(input$num)           # stop silently if input$num is NULL or empty
    hist(rnorm(input$num))
  })
}

```

- When an app first loads, `input` values are `NULL` until the user interacts. Render functions that use those inputs will throw errors during initialization.
- `req(x)` checks that `x` is truthy (not `NULL`, not `NA`, not empty string, not `FALSE`, not a zero-length vector). If the check fails it raises a special condition that Shiny catches silently — the output is left blank.
- `req()` is essential when:
 - An input depends on another input (e.g., a dropdown that populates based on a previous selection).
 - File uploads are read before a file has been chosen.
 - Any dynamic `renderUI()` widget has not yet been created.
- You can pass multiple arguments: `req(input$x, input$y)` — both must be truthy.

Render and Output Functions

Each output type requires a **matching** pair of functions — one in the server (`render*`()), one in the UI (`*Output()`):

Output function	Render function	Displays
<code>plotOutput()</code>	<code>renderPlot()</code>	plots (e.g. <code>ggplot2</code>)
<code>tableOutput()</code>	<code>renderTable()</code>	data frames / matrices
<code>dataTableOutput()</code>	<code>renderDataTable()</code>	interactive DT table
<code>textOutput()</code>	<code>renderText()</code>	plain text values
<code>verbatimTextOutput()</code>	<code>renderPrint()</code>	<code>print()</code> output
<code>imageOutput()</code>	<code>renderImage()</code>	image files (.png, .jpg)
<code>uiOutput()</code>	<code>renderUI()</code>	dynamic Shiny UI
<code>downloadButton()</code>	<code>downloadHandler()</code>	file downloads

- `renderText()` collapses its result into a single string with `paste()`. `renderPrint()` captures everything that would appear in the R console, including invisible values.
- `renderUI()` / `uiOutput()` is the escape hatch for fully dynamic interfaces (e.g., a variable number of inputs depending on the data).

Exercises 1–3: Gapminder App

Setup

```
library(shiny)
library(gapminder)
library(munsell)
library(ggplot2)
library(dplyr)
library(DT)
theme_set(theme_light())
```

Exercise 1 Solution — Basic Scatter Plot

The axes are restricted to the three numeric variables `lifeExp`, `pop`, and `gdpPercap` via `choices`. The `.data[[ ]]` pronoun from tidy evaluation is used to look up column names supplied as plain character strings from `selectInput()`.

```
ui <- fluidPage(
  titlePanel("Gapminder"),
  sidebarLayout(
    sidebarPanel(
      selectInput("xvar", "x-axis",
                  choices = c("lifeExp", "pop", "gdpPercap"),
                  selected = "lifeExp"),
      selectInput("yvar", "y-axis",
                  choices = c("lifeExp", "pop", "gdpPercap"),
                  selected = "gdpPercap")
    ),
    mainPanel(plotOutput("scatterPlot"))
  )
)

server <- function(input, output) {
  output$scatterPlot <- renderPlot({
    ggplot(gapminder, aes(x = .data[[input$xvar]], y = .data[[input$yvar]])) +
      geom_point() +
      expand_limits(x = 0, y = 0)
  })
}

shinyApp(ui, server)
```

- `selectInput()` returns a plain character string, so `.data[[input$xvar]]` is the correct tidy-evaluation pronoun. This is simpler than `varSelectInput()` which returns a symbol and requires `!!`.
- `expand_limits(x = 0, y = 0)` ensures the origin is always visible, providing a stable reference for comparison across selections.

Exercise 2 Solution — Color and Size Controls

Size is now mapped to a data variable (`sizevar`) rather than a fixed slider value, matching the Gapminder bubble-chart style where population drives point size.

```
ui <- fluidPage(
  titlePanel("Gapminder"),
```

```

sidebarLayout(
  sidebarPanel(
    selectInput("xvar", "x-axis",
               choices = names(gapminder)[4:6], selected = "lifeExp"),
    selectInput("yvar", "y-axis",
               choices = names(gapminder)[4:6], selected = "gdpPercap"),
    selectInput("colvar", "color by",
               choices = c("continent", names(gapminder)[4:6]),
               selected = "continent"),
    selectInput("sizevar", "size by",
               choices = names(gapminder)[4:6],
               selected = "pop")
  ),
  mainPanel(plotOutput("scatterPlot"))
)

server <- function(input, output) {
  output$scatterPlot <- renderPlot({
    ggplot(gapminder,
           aes(
             x = .data[[input$xvar]],
             y = .data[[input$yvar]],
             color = .data[[input$colvar]],
             size = .data[[input$sizevar]]
           )
    ) +
    geom_point(alpha = 0.7) + expand_limits(x = 0, y = 0)
  })
}

shinyApp(ui, server)

```

- `names(gapminder)[4:6]` evaluates to `c("lifeExp", "pop", "gdpPercap")` — the three numeric columns. Using index subsetting keeps `choices` automatically correct if the data frame column order is unchanged.
- `alpha = 0.7` reduces overplotting when many points overlap.

Exercise 3 Solution — Year Filter

The year filter is implemented as a `reactive()` conductor so the filtered data frame is computed once and can be reused by multiple outputs as the app grows.

```

ui <- fluidPage(
  titlePanel("Gapminder"),
  sidebarLayout(
    sidebarPanel(
      selectInput("xvar", "x-axis",
                 choices = names(gapminder)[4:6], selected = "lifeExp"),
      selectInput("yvar", "y-axis",
                 choices = names(gapminder)[4:6], selected = "gdpPercap"),
      selectInput("colvar", "color by",
                 choices = c("continent", names(gapminder)[4:6]),
                 selected = "continent"),

```

```

    selectInput("sizevar", "size by",
               choices = names(gapminder)[4:6],
               selected = "pop"),
    sliderInput("year", "year",
               min = min(gapminder$year), max = max(gapminder$year),
               value = min(gapminder$year),
               step = 5, sep = "")
  ),
  mainPanel(plotOutput("scatterPlot"))
)
}

server <- function(input, output) {
  filteredData <- reactive({
    gapminder %>% filter(year == input$year)
  })

  output$scatterPlot <- renderPlot({
    ggplot(filteredData(),
           aes(
             x = .data[[input$xvar]],
             y = .data[[input$yvar]],
             color = .data[[input$colvar]],
             size = .data[[input$sizevar]]
           ))
    +
    geom_point(alpha = 0.7) + expand_limits(x = 0, y = 0) +
    ggtitle(paste("Year:", input$year))
  })
}

shinyApp(ui, server)

```

- `sep = ""` in `sliderInput()` removes the thousands separator from year numbers (no “1,952”).
- `value = min(gapminder$year)` starts the slider at the earliest year (1952) rather than the most recent, so the animation effect of dragging is more natural.
- The `filteredData` reactive conductor makes year-filtering a single reusable unit — later exercises build directly on this pattern.

Reactivity — Three Kinds of Reactive Objects

- **Reactive source** (`input$*`): read-only from the server perspective; always a parent. Changes *invalidate* all dependents.
- **Reactive endpoint** (`output$*`, created by `render*()`): read the reactive graph when a source changes and re-execute. Always a child.
- **Reactive conductor** (`reactive()`): can be both parent and child. Caches its result and re-runs only when its own inputs are invalidated.
- Reactivity in Shiny is *lazy* and *cached*:
 - **Lazy**: a reactive expression does not run until something downstream demands its value.
 - **Cached**: once computed, the value is reused by all consumers until an upstream source changes.

- This is different from Excel-style eager recalculation. Shiny only does work that is actually needed.

Reactivity — A Simple App

```
# reactive graph: input$num -> output$hist
server <- function(input, output, session) {
  output$hist <- renderPlot({
    hist(rnorm(input$num))
  })
}
```

- When `input$num` changes, Shiny marks `output$hist` as *invalidated*.
- On the next render cycle, Shiny re-runs the body of `renderPlot()` and sends the new image to the browser.
- If `output$hist` is not currently visible (e.g., behind a tab panel), Shiny skips re-execution until it becomes visible.

Reactivity — Avoiding Redundant Computation

```
fib <- function(n) ifelse(n < 3, 1, fib(n-1) + fib(n-2))

server <- function(input, output, session) {
  output$nthValue <- renderText({ fib(as.numeric(input$n)) })
  output$nthValueInv <- renderText({ 1 / fib(as.numeric(input$n)) })
}
```

- Both `renderText()` calls read `input$n` and call `fib()` independently.
- When `input$n` changes, Shiny invalidates *both* endpoints and each recomputes `fib()` from scratch. For large `n`, the exponential recursion is called twice unnecessarily.
- This is the pattern a reactive conductor solves.

Reactive Conductors with `reactive()`

```
server <- function(input, output, session) {
  currentFib <- reactive({ fib(as.numeric(input$n)) })
  output$nthValue <- renderText({ currentFib() })
  output$nthValueInv <- renderText({ 1 / currentFib() })
}
```

- `reactive()` creates a reactive conductor that caches its result.
- Both `renderText()` calls call `currentFib()` but `fib()` runs **only once** — Shiny serves the cached value to the second caller.
- **Syntax detail:** access a reactive conductor with `currentFib()`, *not* `currentFib`. The parentheses signal to Shiny that you are reading a reactive value and establish the dependency.
- **Common mistake:** calling `currentFib` without parentheses returns the reactive object itself, not its value. This silently breaks the reactive graph.
- `reactive()` conductors also help with code clarity: give the intermediate result a meaningful name rather than embedding complex logic inside multiple render functions.

Triggering Reactions on Demand

The problem with numeric inputs:

```
# This re-runs on every single keystroke while typing "1", "10", "100"
output$hist <- renderPlot({
  hist(rnorm(input$n, mean = input$mean, sd = input$sd))
})
```

- Each keystroke in a `numericInput` fires a new reactive invalidation, causing the plot to flicker and possibly triggering expensive computations mid-entry.

`eventReactive()` — delay computation until a button is clicked:

```
server <- function(input, output, session) {
  x <- eventReactive(input$goButton, {
    rnorm(input$n, mean = input$mean, sd = input$sd)
  })
  output$hist <- renderPlot({ hist(x()) })
}
```

- `eventReactive(event, expr)` is a reactive conductor that only re-executes `expr` when `event` fires (e.g., a button click), regardless of what other inputs change.
- Internally it uses `isolate()` to wrap `expr` so that reading `input$n`, `input$mean`, `input$sd` inside it does *not* create dependencies. Only `input$goButton` is a dependency.
- Access the value with `x()` (parentheses required, as with `reactive()`).

`observeEvent()` — side effects without a return value:

```
observeEvent(input$goButton, {
  cat("Button clicked, n =", input$n, "\n")
})
```

- Use `observeEvent()` when you want to *do* something (write a file, log a value, update another input) but do not need to produce output for the UI.
- `observeEvent()` returns invisibly. Do not assign its result to `output$*`.
- Rule of thumb: `eventReactive()` when you need a value; `observeEvent()` when you need a side effect.

Full `eventReactive()` Example

```
ui <- fluidPage(
  numericInput("n", "Number of observations", 100),
  numericInput("mean", "Mean", 0),
  numericInput("sd", "Standard Deviation", 1),
  actionButton("goButton", "Go!"),
  plotOutput("hist")
)

server <- function(input, output, session) {
  x <- eventReactive(input$goButton, {
    rnorm(input$n, mean = input$mean, sd = input$sd)
  })
  output$hist <- renderPlot({ hist(x()) })
}

shinyApp(ui, server)
```

- Note that `output$hist` depends on `x()`, which only changes when the button is clicked. The plot does not update while the user is still typing.
- **`isolate()` as a lower-level alternative:** wraps a reactive expression to suppress dependency creation for that specific read. `eventReactive()` is the idiomatic pattern; prefer it over manually placing `isolate()`.

```
# lower-level, less readable:
output$hist <- renderPlot({
  input$goButton # establish dependency only on button
  isolate(hist(rnorm(input$n, mean = input$mean, sd = input$sd)))
})
```

Exercises 4–5: Gapminder App (cont.)

Exercise 4 Solution — Action Button with `eventReactive()`

```
ui <- fluidPage(
  titlePanel("Gapminder"),
  sidebarLayout(
    sidebarPanel(
      selectInput("xvar", "x-axis",
        choices = names(gapminder)[4:6], selected = "lifeExp"),
      selectInput("yvar", "y-axis",
        choices = names(gapminder)[4:6], selected = "gdpPercap"),
      selectInput("colvar", "color by",
        choices = c("continent", names(gapminder)[4:6]),
        selected = "continent"),
      selectInput("sizevar", "size by",
        choices = names(gapminder)[4:6],
        selected = "pop"),
      sliderInput("year", "year",
        min = min(gapminder$year),
        max = max(gapminder$year),
        value = min(gapminder$year),
        step = 5,
        sep = ""
      ),
      actionButton("goBtn", "Update Plot")
    ),
    mainPanel(plotOutput("scatterPlot"))
  )
)

server <- function(input, output) {
  dataReactive <- eventReactive(input$goBtn, {
    gapminder %>% filter(year == input$year)
  })

  output$scatterPlot <- renderPlot({
    df <- dataReactive()
    ggplot(df, aes(
      x      = .data[[input$xvar]],
```

```

    y      = .data[[input$yvar]],
    color  = .data[[input$colvar]],
    size   = .data[[input$sizevar]]
  )) +
  geom_point(alpha = 0.7) + expand_limits(x = 0, y = 0) +
  ggtitle(paste("Year:", input$year))
})
}

shinyApp(ui, server)

```

- The reactive is named `dataReactive` to make its role explicit: it encapsulates the expensive filtering step and only re-runs when the button is clicked.
- Note that `input$xvar`, `input$yvar`, `input$colvar`, and `input$sizevar` are read inside `renderPlot()` — not inside `eventReactive()` — so aesthetic changes (axis, color, size variable) take effect immediately without a button click. Only the year filter is gated.

Exercise 5 Solution — Continent Aggregation with `reactive()`

Two reactive layers are used: `aggData` pre-computes the continent-level summary (and is only triggered when the checkbox is ticked), while `plotData` gates the full render behind the action button.

```

ui <- fluidPage(
  titlePanel("Gapminder"),
  sidebarLayout(
    sidebarPanel(
      selectInput("xvar", "x-axis",
        choices = names(gapminder)[4:6], selected = "lifeExp"),
      selectInput("yvar", "y-axis",
        choices = names(gapminder)[4:6], selected = "gdpPercap"),
      selectInput("colvar", "color by",
        choices = c("continent", names(gapminder)[4:6]),
        selected = "continent"),
      selectInput("sizevar", "size by",
        choices = names(gapminder)[4:6],
        selected = "pop"),
      sliderInput("year", "year",
        min = min(gapminder$year),
        max = max(gapminder$year),
        value = min(gapminder$year),
        step = 5,
        sep = ""
      ),
      checkboxInput("aggContinent", "Only continents", FALSE),
      actionButton("goBtn", "Update Plot")
    ),
    mainPanel(
      plotOutput("scatterPlot")
    )
  )
)

server <- function(input, output) {
  aggData <- reactive({

```

```

    req(input$aggContinent)
    gapminder %>%
      group_by(continent, year) %>%
      summarise(lifeExp = mean(lifeExp), pop = sum(pop),
                gdpPercap = mean(gdpPercap), .groups = "drop")
  })

plotData <- eventReactive(input$goBtn, {
  df <- if (input$aggContinent) aggData() else gapminder
  df <- df %>% filter(year == input$year)
  title <- if (input$aggContinent)
    paste("Continent summary, Year:", input$year)
  else paste("Year:", input$year)
  list(df = df, title = title)
}, ignoreNULL = FALSE)

output$scatterPlot <- renderPlot({
  pd <- plotData()
  ggplot(pd$df, aes(
    x = .data[[input$xvar]],
    y = .data[[input$yvar]],
    color = .data[[input$colvar]],
    size = .data[[input$sizevar]]
  )) +
    geom_point(alpha = 0.7) +
    expand_limits(x = 0, y = 0) +
    ggtitle(pd$title)
})
}

shinyApp(ui, server)

```

- `aggData` uses `req(input$aggContinent)` — it only computes the summary when the checkbox is TRUE. When unchecked, `aggData()` is never called and no grouping computation occurs.
- `plotData` returns a named `list(df, title)` so the server function has a single reactive source for both the data and the plot title, avoiding duplication.
- `ignoreNULL = FALSE` in `eventReactive()` causes the app to render the initial plot on startup without requiring a button click.

DataTables

```

library(DT)

ui <- basicPage(
  h2("The mtcars data"),
  DT::dataTableOutput("mytable")
)

server <- function(input, output, session) {
  output$mytable <- DT::renderDataTable(
    mtcars,
    filter = "top"
  )
}

```

```
)
}

shinyApp(ui, server)
```

- Use `DT::` prefix explicitly. Base Shiny also has `dataTableOutput()` / `renderDataTable()` (an older, simpler version). Namespacing avoids silent conflicts.
- `filter = "top"` adds a per-column filter row above the table header. `filter = "bottom"` places it below.
- `DT::datatable()` inside `renderDataTable()` gives full control over options (page length, column formatting, row selection, button extensions, etc.):

```
output$mytable <- DT::renderDataTable({
  DT::datatable(
    mtcars,
    filter      = "top",
    selection   = "multiple",
    options     = list(pageLength = 10, scrollX = TRUE)
  )
})
```

DataTables — Using Filtered Rows

```
ui <- basicPage(
  h2("The mtcars data"),
  DT::dataTableOutput("mytable"),
  plotOutput("hist")
)

server <- function(input, output, session) {
  output$mytable <- DT::renderDataTable(
    mtcars,
    filter = "top"
  )
  output$hist <- renderPlot({
    rows <- input$mytable_rows_all
    req(rows)
    hist(mtcars[rows, "wt"], xlim = range(mtcars$wt))
  })
}

shinyApp(ui, server)
```

- `input$mytable_rows_all` is the integer vector of *all* row indices that pass the current filter (across all pages). It is a reactive value maintained by the DT widget.
- `input$mytable_rows_selected` contains only the rows the user has *clicked* to select (requires `selection = "multiple"` or `"single"`).
- `xlim = range(mtcars$wt)` keeps the axis fixed so the histogram scale does not jump when the filter changes — important for visual continuity.
- `req(rows)` guards against the initial state where no filter has been applied yet and `rows` may be `NULL`.

- **Reactive graph:** `input$mytable_rows_all` acts as a reactive source just like any other input. The histogram re-renders automatically when the user edits a column filter.

Exercise 6 Solution — Tab Panel with DataTable

This is the full cumulative app. The `baseData` reactive handles the optional continent aggregation; `plotData` gates everything behind the button and returns a list consumed by both outputs.

```
ui <- fluidPage(
  titlePanel("Gapminder"),
  sidebarLayout(
    sidebarPanel(
      selectInput("xvar", "x-axis",
        choices = names(gapminder)[4:6], selected = "lifeExp"),
      selectInput("yvar", "y-axis",
        choices = names(gapminder)[4:6], selected = "gdpPercap"),
      selectInput("colvar", "color by",
        choices = c("continent", names(gapminder)[4:6]),
        selected = "continent"),
      selectInput("sizevar", "size by",
        choices = names(gapminder)[4:6],
        selected = "pop"),
      sliderInput("year", "year",
        min = min(gapminder$year),
        max = max(gapminder$year),
        value = min(gapminder$year),
        step = 5,
        sep = ""
      ),
      checkboxInput("aggContinent", "Only continents", FALSE),
      actionButton("goBtn", "Update")
    ),
    mainPanel(
      tabsetPanel(
        tabPanel("Plot", plotOutput("scatterPlot")),
        tabPanel("Table", DT::dataTableOutput("dataTbl"))
      )
    )
  )
)

server <- function(input, output) {
  baseData <- reactive({
    if (input$aggContinent) {
      gapminder %>%
        group_by(continent, year) %>%
        summarise(
          lifeExp = mean(lifeExp),
          pop     = sum(pop),
          gdpPercap = mean(gdpPercap),
          .groups = "drop"
        )
    } else gapminder
  })
}
```

```

plotData <- eventReactive(input$goBtn, {
  df <- baseData() %>% filter(year == input$year)
  title <- if (input$aggContinent)
    paste("Continent summary, Year:", input$year)
  else paste("Year:", input$year)
  list(df = df, title = title)
},
ignoreNULL = FALSE
)

output$scatterPlot <- renderPlot({
  pd <- plotData()
  ggplot(pd$df, aes(
    x      = .data[[input$xvar]],
    y      = .data[[input$yvar]],
    color  = .data[[input$colvar]],
    size   = .data[[input$sizevar]]
  )) +
    geom_point(alpha = 0.7) +
    expand_limits(x = 0, y = 0) + ggtitle(pd$title)
})

output$dataTbl <- DT::renderDataTable({
  plotData()$df
}, options = list(pageLength = 10))
}

shinyApp(ui, server)

```

- `baseData` is a `reactive()` (not `eventReactive()`) so the aggregation toggle takes effect as soon as the checkbox changes, making the data layer responsive independently of the button.
- `plotData()`\$`df` is shared between the plot and the table: both consume the same already-filtered data frame — no duplicate filtering.
- **Lazy rendering:** Shiny does not execute `output$dataTbl` until the user switches to the “Table” tab. This is a free performance win for expensive table renders.
- `options = list(pageLength = 10)` passed directly to `DT::renderDataTable()` sets the default page size without wrapping in `DT::datatable()`.